

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - CƠ - TIN HỌC



BÁO CÁO BÀI TẬP LỚN

**Tìm hiểu các phương pháp giải chính xác cho
bài toán định tuyến phương tiện có ràng
buộc khung thời gian (VRPTW)**

Sinh viên thực hiện	Mã sinh viên
Trần Đức Kiên (Leader)	23000131
Tạ Văn Nghĩa	23000145
Vũ Tiến Đạt	23000111

Hà Nội, 25 - 4 - 2026

Mục lục

GIỚI THIỆU CHUNG	1
CHƯƠNG I: TỔNG QUAN BÀI TOÁN	2
1 MÔ TẢ BÀI TOÁN	2
1.1 Bài toán VRP	2
1.2 Bài toán VRPTW	2
1.3 Input	3
1.4 Output	4
1.5 Các phương pháp giải chính xác cho bài toán VRPTW	5
1.6 Bộ dữ liệu của Solomon Benchmark	5
2 MÔ HÌNH HÓA BÀI TOÁN	6
2.1 Tập hợp và tham số	6
2.2 Biến quyết định	7
2.3 Hàm mục tiêu	7
2.4 Các ràng buộc	7
CHƯƠNG II: CÁC PHƯƠNG PHÁP GIẢI CHÍNH XÁC	9
1 BRANCH AND BOUND	9
1.1 Khái quát	9
1.2 Ý tưởng thuật toán	9
1.3 Pseudocode	12
1.4 Độ phức tạp của thuật toán	13
2 BRANCH AND CUT	14
2.1 Khái quát	14
2.2 Bất đẳng thức hợp lệ (Valid Inequalities)	14
2.2.1 Capacity Inequalities	14
2.2.2 2-Cycle Elimination Inequalities	15
2.3 Thuật toán co cụm đồ thị (Graph Shrinking)	16
2.4 Sơ đồ thuật toán Branch-and-Cut	18

2.4.1	Đánh giá độ phức tạp	19
3	BRANCH AND PRICE	20
3.1	Khái quát	20
3.2	Khởi tạo	21
3.3	Column Generation	22
3.3.1	Restricted Master Problem (RMP)	22
3.3.2	Bài toán con: Cập nhật reduced cost và gọi SPPRC	23
3.4	Bài toán con SPPRC	24
3.5	Branch and Bound trong Branch and Price	28
3.6	Luồng thực thi tổng thể	32
3.7	Độ phức tạp	33
	CHƯƠNG III: KẾT QUẢ THỰC NGHIỆM	35
1	Kết quả thực nghiệm	35
1.1	C101 (Phân cụm)	36
1.2	R101 (Ngẫu nhiên)	37
1.3	RC101 (Hỗn hợp)	39
2	Đánh giá chi tiết	40
	KẾT LUẬN	43
	Tài liệu tham khảo	44

Danh sách hình vẽ

1	Minh họa quá trình co cụm đồ thị	17
2	Biểu đồ so sánh thời gian chạy của các thuật toán (C101)	36
3	Sơ đồ lộ trình tối ưu tại mốc $n = 11$ (C101)	37
4	Biểu đồ so sánh thời gian chạy của các thuật toán (R101)	38
5	Sơ đồ lộ trình tối ưu tại mốc $n = 11$ (R101)	38
6	Biểu đồ so sánh thời gian chạy của các thuật toán (RC101)	39
7	Sơ đồ lộ trình tối ưu tại mốc $n = 11$ (RC101)	40

Danh sách bảng

1	Đánh giá hiệu năng tổng hợp của các thuật toán trên bộ dữ liệu C101	36
2	Đánh giá hiệu năng tổng hợp của các thuật toán trên bộ dữ liệu R101	37
3	Đánh giá hiệu năng tổng hợp của các thuật toán trên bộ dữ liệu RC101	39
4	Tổng kết so sánh đặc điểm vận hành của B&B, B&C và B&P	42

GIỚI THIỆU CHUNG

Bối cảnh và đặt vấn đề

Trong bối cảnh ngành logistics và vận tải ngày càng phát triển, nhu cầu tối ưu hóa việc giao nhận hàng hóa trở nên đặc biệt quan trọng nhằm giảm chi phí vận hành, tiết kiệm thời gian và nâng cao chất lượng dịch vụ. Bài toán định tuyến phương tiện có ràng buộc khung thời gian (Vehicle Routing Problem with Time Windows – VRPTW) là một trong những bài toán tối ưu tổ hợp quan trọng trong lĩnh vực Operations Research và logistics.

VRPTW không chỉ yêu cầu xác định lộ trình tối ưu cho các phương tiện mà còn phải đảm bảo mỗi khách hàng được phục vụ trong khoảng thời gian quy định. Đây là bài toán có tính ứng dụng cao, độ phức tạp lớn thuộc lớp NP-hard. Vì vậy, việc tìm hiểu các phương pháp giải chính xác cho bài toán VRPTW có ý nghĩa quan trọng cả về mặt lý thuyết và ứng dụng thực tiễn.

Việc giải quyết bài toán VRPTW giúp:

- Giảm tổng quãng đường di chuyển
- Giảm chi phí nhiên liệu
- Tối ưu số lượng phương tiện sử dụng

Mục tiêu

Mục tiêu của bài báo cáo này là tìm hiểu tổng quan về bài toán định tuyến phương tiện có ràng buộc khung thời gian (Vehicle Routing Problem with Time Windows – VRPTW). Bên cạnh đó, xây dựng mô hình toán học cho bài toán VRPTW, bao gồm hàm mục tiêu, biến quyết định và các ràng buộc liên quan đến lộ trình và khung thời gian phục vụ.

Ngoài ra, báo cáo sẽ sử dụng một số phương pháp giải chính xác phổ biến được áp dụng cho bài toán VRPTW như Branch and Bound, Branch and Cut, Branch and Price và Branch and Cut and Price. Nhóm thực nghiệm trên bộ dữ liệu Solomon Benchmark thường được sử dụng để đánh giá hiệu quả của các thuật toán giải VRPTW. Thông qua đó, báo cáo phân tích và đánh giá ưu điểm, hạn chế của các phương pháp giải chính xác đối với bài toán này.

CHƯƠNG I: TỔNG QUAN BÀI TOÁN

1 MÔ TẢ BÀI TOÁN

1.1 Bài toán VRP

Bài toán Định tuyến Phương tiện (Vehicle Routing Problem - VRP) là dạng tổng quát hóa của bài toán Người bán hàng (TSP), hướng tới việc xác định lộ trình di chuyển tối ưu cho một đội xe nhằm giảm thiểu chi phí hoặc thời gian giao hàng. Được Dantzig và Ramser giới thiệu lần đầu năm 1959 qua bài toán vận chuyển xăng dầu, VRP thuộc lớp tối ưu tổ hợp NP-khó, nghĩa là hiện chưa có thuật toán giải chính xác trong thời gian đa thức. Ngày nay, trước sự bùng nổ của thương mại điện tử, VRP ngày càng thu hút nhiều sự quan tâm nghiên cứu. Việc giải quyết hiệu quả bài toán này đóng vai trò cốt lõi giúp các doanh nghiệp vận tải tối đa hóa hiệu suất đội xe và cắt giảm tối đa chi phí vận hành.

1.2 Bài toán VRPTW

Bài toán Định tuyến Phương tiện có Ràng buộc Khung thời gian (Vehicle Routing Problem with Time Windows - VRPTW) là một biến thể mở rộng quan trọng của bài toán VRP. Ngoài mục tiêu tối ưu hóa lộ trình di chuyển của đội xe, bài toán còn yêu cầu mỗi khách hàng phải được phục vụ trong một khoảng thời gian xác định trước, gọi là khung thời gian (Time Window).

Bài toán được định nghĩa bởi một đội xe K , một tập khách hàng V và một đồ thị có hướng G . Đội xe được coi là đồng nhất (tất cả các xe đều giống hệt nhau). Đồ thị bao gồm $|C| + 1$ đỉnh, trong đó các khách hàng được đánh số là $1, 2, \dots, n$ và kho bãi (depot) đánh số là 0 . Tập hợp tất cả các đỉnh $0, 1, \dots, n$ được ký hiệu là N .

$A = \{(i, j) : i, j \in V, i \neq j\}$ là tập hợp các tuyến đường khả thi giữa các đỉnh trong đồ thị. Với mỗi cung (i, j) , tồn tại một chi phí di chuyển c_{ij} và thời gian di chuyển tương ứng t_{ij} .

Mỗi xe có sức chứa là Q và mỗi khách hàng i có một nhu cầu hàng hóa q_i . Mỗi khách hàng i có một khung thời gian $[a_i, b_i]$, trong đó xe phải bắt đầu phục vụ khách hàng trước thời điểm b_i . Nếu xe đến trước thời điểm mở cửa a_i , xe phải chờ đến thời điểm a_i mới được phép bắt đầu phục vụ.

Khung thời gian của kho(depôt) được ký hiệu là $[a_0, b_0]$, đại diện cho khoảng thời gian

hoạt động của toàn bộ hệ thống giao hàng. Các phương tiện không được phép rời kho(depot) trước thời điểm a_0 và phải quay trở lại depot không muộn hơn thời điểm b_0 .

Có hai mục tiêu chính thường được tối ưu hóa trong bài toán VRPTW là tối thiểu hóa số lượng xe cần sử dụng để phục vụ toàn bộ khách hàng, tối thiểu hóa tổng chi phí di chuyển.

1.3 Input

Bài toán định tuyến xe có cửa sổ thời gian (VRPTW) được mô hình hóa trên một đồ thị có hướng:

$$G = (V, A)$$

trong đó:

$$V = \{0, 1, 2, \dots, n\}$$

là tập các đỉnh. Đỉnh 0 biểu diễn depot, còn

$$N = \{1, 2, \dots, n\}$$

là tập các khách hàng. Tập cung được định nghĩa bởi:

$$A = \{(i, j) \mid i, j \in V, i \neq j\}$$

Với mỗi cung $(i, j) \in A$, bài toán cho trước chi phí di chuyển $c_{ij} \geq 0$ và thời gian di chuyển $t_{ij} \geq 0$. Mỗi khách hàng $i \in N$ được đặc trưng bởi các tham số:

$$q_i, [a_i, b_i], s_i$$

trong đó q_i là nhu cầu của khách hàng, $[a_i, b_i]$ là cửa sổ thời gian cho phép phục vụ, và s_i là thời gian phục vụ tại khách hàng i . Depot cũng có cửa sổ thời gian riêng, ký hiệu là:

$$[a_0, b_0]$$

Đội xe được ký hiệu là:

$$K = \{1, 2, \dots, m\}$$

với các xe được giả định là đồng nhất và có sức chứa tối đa:

$$Q$$

Số lượng xe ký hiệu là:

$$m$$

Do đó, đầu vào của bài toán VRPTW có thể được tóm tắt như sau:

$$\textbf{Input VRPTW} = G(V, A), c_{ij}, t_{ij}, q_i, [a_i, b_i], s_i, Q, K, m$$

Trong bộ dữ liệu Solomon, mỗi điểm $i \in V$ được cung cấp tọa độ (x_i, y_i) . Khi đó, chi phí hoặc thời gian di chuyển giữa hai điểm có thể được tính bằng khoảng cách Euclidean:

$$c_{ij} = t_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

với giả định vận tốc di chuyển được chuẩn hóa bằng 1.

1.4 Output

Đầu ra của bài toán VRPTW là một tập các tuyến đường khả thi:

$$R^* = \{R_1^*, R_2^*, \dots, R_{K^*}^*\}$$

trong đó mỗi tuyến bắt đầu từ depot, phục vụ một số khách hàng theo một thứ tự xác định và quay trở lại depot:

$$R_k^* = (0, i_1, i_2, \dots, i_p, 0)$$

Các tuyến đường phải thỏa mãn các ràng buộc: mỗi khách hàng được phục vụ đúng một lần, tổng nhu cầu trên mỗi tuyến không vượt quá sức chứa xe Q , và thời điểm bắt đầu phục vụ tại mỗi khách hàng i thuộc cửa sổ thời gian $[a_i, b_i]$.

Với biến quyết định:

$$x_{ij} = \begin{cases} 1, & \text{nếu có một xe di chuyển trực tiếp từ đỉnh } i \text{ đến đỉnh } j, \\ 0, & \text{ngược lại,} \end{cases}$$

mục tiêu của bài toán là tìm tập tuyến có tổng chi phí di chuyển nhỏ nhất:

$$\min \sum_{i \in V} \sum_{\substack{j \in V \\ j \neq i}} c_{ij} x_{ij}$$

Ngoài tập tuyến tối ưu, nghiệm của bài toán còn xác định thời điểm bắt đầu phục vụ tại mỗi khách hàng:

$$T_i^*, \quad i \in N$$

Do đó, đầu ra của bài toán có thể được tóm tắt là:

$$\text{Output VRPTW} = R^*, x_{ij}^*, T_i^*, Z^*$$

trong đó:

$$Z^* = \sum_{i \in V} \sum_{\substack{j \in V \\ j \neq i}} c_{ij} x_{ij}^*$$

là tổng chi phí di chuyển nhỏ nhất.

1.5 Các phương pháp giải chính xác cho bài toán VRPTW

Do bài toán VRPTW thuộc lớp bài toán NP-khó, việc tìm nghiệm tối ưu cho các bài toán có kích thước lớn là một thách thức đáng kể. Để giải quyết bài toán này, nhiều phương pháp giải chính xác đã được đề xuất nhằm đảm bảo tìm được nghiệm tối ưu toàn cục.

Các phương pháp giải chính xác cho bài toán VRPTW trong bài báo cáo này bao gồm:

- **Branch and Bound:** Phương pháp phân chia không gian nghiệm thành các bài toán con dưới dạng cây tìm kiếm, đồng thời sử dụng cận trên và cận dưới để loại bỏ các nhánh không tiềm năng.
- **Branch and Cut:** Là sự kết hợp giữa Branch and Bound và các kỹ thuật Cutting Planes nhằm loại bỏ các nghiệm không hợp lệ và tăng tốc quá trình hội tụ.
- **Branch and Price:** Kết hợp giữa Branch and Bound và kỹ thuật Column Generation, thường được sử dụng cho các mô hình có số lượng biến rất lớn như bài toán VRPTW.

1.6 Bộ dữ liệu của Solomon Benchmark

Trong báo cáo này, nhóm sử dụng bộ dữ liệu Solomon benchmark để thực nghiệm và đánh giá các phương pháp giải chính xác bài toán VRPTW. Đây là bộ dữ liệu chuẩn được đề xuất bởi Solomon vào năm 1987 và được sử dụng rộng rãi trong nhiều nghiên cứu về bài toán định tuyến phương tiện có ràng buộc khung thời gian.

Bộ dữ liệu Solomon gồm nhiều bộ test khác nhau, mỗi bộ bao gồm:

- Một depot trung tâm.
- Tập các khách hàng với tọa độ vị trí trong mặt phẳng.

- Nhu cầu hàng hóa của từng khách hàng.
- Khung thời gian phục vụ (time window).
- Thời gian phục vụ tại mỗi khách hàng.
- Số lượng phương tiện và sức chứa tối đa của mỗi xe.

Các bộ dữ liệu trong Solomon Benchmark được chia thành ba nhóm chính:

- **C (Clustered)**: Các khách hàng được phân bố thành từng cụm.
- **R (Random)**: Các khách hàng được phân bố ngẫu nhiên.
- **RC (Random-Clustered)**: Kết hợp giữa phân bố cụm và ngẫu nhiên.

Bộ dữ liệu Solomon đã được sử dụng và xem như tiêu chuẩn để kiểm tra về hiệu suất, chất lượng của thuật toán giải quyết bài toán VRP. Các nhà nghiên cứu thường dùng bộ dữ liệu trên cũng như các phiên bản mở rộng của nó để so sánh, đánh giá các thuật toán khác nhau.

2 MÔ HÌNH HÓA BÀI TOÁN

Quy hoạch Tuyến tính Nguyên hỗn hợp (MILP) với biến liên tục phụ trợ Đây là cách mô hình hóa phổ biến được Bard, Kontoravdis và Yu đề xuất, sử dụng biến nhị phân x_{ij} kết hợp với các biến liên tục gắn tại từng khách hàng để theo dõi thời gian phục vụ và tải trọng của xe. [1]

2.1 Tập hợp và tham số

- $I = \{1, 2, \dots, n\}$: tập hợp các khách hàng.
- 0: điểm tập kết kho, hay depot.
- $I_0 = I \cup \{0\}$: tập hợp tất cả các nút, bao gồm khách hàng và depot.
- q_i : nhu cầu của khách hàng i , với $q_i > 0$, $\forall i \in I$.
- Q : sức chứa tối đa của một phương tiện.

- m : số lượng phương tiện tối đa có thể sử dụng.
- $[a_i, b_i]$: cửa sổ thời gian của khách hàng i , trong đó a_i là thời điểm sớm nhất và b_i là thời điểm muộn nhất được phép phục vụ.
- τ_{ij} : thời gian di chuyển từ nút i đến nút j , đã bao gồm thời gian phục vụ tại nút i , với $i, j \in I_0, i \neq j$.

Lưu ý: ta sẽ giả sử tốc độ di chuyển của xe mặc định là 1, nghĩa là với $t_{ij} = 1$ đơn vị thời gian thì xe sẽ di chuyển được $c_{ij} = 1$ đơn vị khoảng cách. Do đó, $c_{ij} = t_{ij} \quad \forall i, j$.

2.2 Biến quyết định

- $x_{ij} \in \{0, 1\}$: biến nhị phân, với $i, j \in I_0, i \neq j$. Biến này được xác định bởi:

$$x_{ij} = \begin{cases} 1, & \text{nếu có xe di chuyển trực tiếp trên cung } (i, j), \\ 0, & \text{ngược lại.} \end{cases}$$

- $t_i \geq 0$: biến liên tục biểu diễn thời điểm xe rời khỏi khách hàng i , với $i \in I_0$.
- $y_i \geq 0$: biến liên tục biểu diễn tổng tải trọng trên xe ngay sau khi rời khỏi khách hàng i , với $i \in I_0$.

2.3 Hàm mục tiêu

Mục tiêu của mô hình là tối thiểu hóa tổng chi phí di chuyển của toàn bộ các tuyến xe:

$$\min Z = \sum_{i \in I_0} \sum_{\substack{j \in I_0 \\ j \neq i}} c_{ij} x_{ij}$$

trong đó c_{ij} là chi phí di chuyển từ nút i đến nút j , và x_{ij} là biến nhị phân xác định cung (i, j) có được chọn hay không.

2.4 Các ràng buộc

- **Ràng buộc phân bổ khách hàng:** Mỗi khách hàng phải được phục vụ đúng một lần.

$$\sum_{\substack{j \in I_0 \\ j \neq i}} x_{ij} = 1, \quad \forall i \in I$$

- **Ràng buộc bảo toàn luồng:** Nếu xe đi vào một khách hàng thì phải rời khỏi khách hàng đó.

$$\sum_{\substack{j \in I_0 \\ j \neq i}} x_{ij} - \sum_{\substack{j \in I_0 \\ j \neq i}} x_{ji} = 0, \quad \forall i \in I$$

- **Ràng buộc liên kết thời gian:** Nếu xe di chuyển từ i đến j , thời gian rời khỏi j phải không nhỏ hơn thời gian rời khỏi i cộng với thời gian di chuyển.

$$t_j \geq t_i + \tau_{ij} - T_{ij}(1 - x_{ij}), \quad \forall i, j \in I, i \neq j$$

trong đó:

$$T_{ij} = b_i + \tau_{ij} - a_j$$

Ràng buộc này đồng thời giúp loại bỏ các chu trình con không đi qua depot do thời gian phải tăng dần dọc theo tuyến.

- **Ràng buộc liên kết sức chứa:** Tải trọng tích lũy sau khi rời khỏi khách hàng j phải tăng thêm đúng bằng nhu cầu của khách hàng j nếu xe đi từ i đến j .

$$y_j \geq y_i + q_j - Q_j(1 - x_{ij}), \quad \forall i, j \in I, i \neq j$$

trong đó:

$$Q_j = Q - q_j$$

- **Ràng buộc cửa sổ thời gian:** Thời điểm rời khỏi mỗi khách hàng phải nằm trong cửa sổ thời gian cho phép.

$$a_i \leq t_i \leq b_i, \quad \forall i \in I$$

- **Ràng buộc tải trọng:** Tải trọng trên xe không được vượt quá sức chứa tối đa.

$$0 \leq y_i \leq Q, \quad \forall i \in I_0.$$

- **Ràng buộc số lượng phương tiện:** Số tuyến xuất phát từ depot không vượt quá số lượng xe cho phép.

$$\sum_{j \in I} x_{0j} \leq m$$

CHƯƠNG II: CÁC PHƯƠNG PHÁP GIẢI CHÍNH XÁC

1 BRANCH AND BOUND

1.1 Khái quát

Branch and Bound (Nhánh và Cận) là một phương pháp giải chính xác phổ biến cho các bài toán tối ưu tổ hợp và quy hoạch nguyên. Ý tưởng chính của Branch and Bound là phân chia bài toán ban đầu thành nhiều bài toán con nhỏ hơn dưới dạng cây tìm kiếm (*Branching*). Với mỗi bài toán con, thuật toán sẽ tính toán các cận trên hoặc cận dưới (*Bounding*) nhằm đánh giá khả năng chứa nghiệm tối ưu của nhánh đó. Nếu một nhánh được xác định chắc chắn không thể tạo ra nghiệm tốt hơn nghiệm hiện tại, nhánh đó sẽ bị loại bỏ khỏi quá trình tìm kiếm (*Pruning*).

Phương pháp Branch and Bound thường bắt đầu bằng việc giải bài toán thư giãn tuyến tính (LP Relaxation). Nếu nghiệm thu được là nghiệm nguyên thì đây chính là nghiệm tối ưu của bài toán. Ngược lại, nếu nghiệm chưa nguyên, thuật toán sẽ lựa chọn một biến phân số để thực hiện phân nhánh, từ đó tạo ra các bài toán con mới với các ràng buộc bổ sung.

1.2 Ý tưởng thuật toán

Thuật toán Branch and Bound hoạt động dựa trên ý tưởng phân chia không gian nghiệm của bài toán thành nhiều bài toán con nhỏ hơn dưới dạng cây tìm kiếm. Mỗi nút trên cây đại diện cho một tập nghiệm khả thi của bài toán.

Đối với bài toán VRPTW, nếu thực hiện vét cạn toàn bộ các phương án phân công xe và lộ trình di chuyển thì số lượng nghiệm sẽ tăng rất nhanh theo cấp số mũ khi số khách hàng tăng lên. Điều này khiến việc tìm kiếm trực tiếp nghiệm tối ưu trở nên không khả thi đối với các bộ dữ liệu thực tế. Vì vậy, Branch and Bound được sử dụng nhằm giảm không gian tìm kiếm bằng cách loại bỏ sớm các nhánh không tiềm năng.

Ý tưởng của thuật toán có thể được mô tả qua các bước sau:

1. Khởi tạo.

Thiết lập tập các nút cần duyệt:

$$\Omega = \{\omega_0\}$$

trong đó ω_0 là nút gốc tương ứng với bài toán VRPTW ban đầu. Khởi tạo nghiệm nguyên tốt nhất hiện tại:

$$Z^* = +\infty.$$

Nếu có nghiệm khả thi ban đầu từ một thuật toán heuristic, ta có thể dùng giá trị đó làm cận trên ban đầu.

2. Đánh giá cận bằng nói lỏng tuyến tính.

Tại mỗi nút $\omega \in \Omega$, ta giải bài toán nói lỏng tuyến tính bằng cách thay ràng buộc nhị phân:

$$x_{ij} \in \{0, 1\}$$

bởi:

$$0 \leq x_{ij} \leq 1.$$

Bài toán LP thu được cho giá trị mục tiêu:

$$Z_{LP}(\omega)$$

đây là cận dưới của nút ω đối với bài toán tối thiểu hóa.

3. Cắt tĩa nút.

Sau khi giải bài toán nói lỏng LP tại nút ω , thuật toán loại bỏ nút này nếu một trong các điều kiện sau xảy ra:

- **Vô nghiệm:** bài toán LP tại nút ω không có nghiệm khả thi.
- **Cận dưới không tốt hơn nghiệm hiện tại:**

$$Z_{LP}(\omega) \geq Z^*.$$

- **Nghiệm nguyên:** nếu tất cả các biến x_{ij} trong nghiệm LP đều nhận giá trị 0 hoặc 1, thì nghiệm này là nghiệm khả thi của bài toán MILP. Khi đó cập nhật:

$$Z^* = \min\{Z^*, Z_{LP}(\omega)\}.$$

4. Phân nhánh.

Khi nghiệm $\bar{x}$ của $LP(\omega)$ chưa nguyên và $LB(\omega) < Z^*$, thuật toán tiến hành rẽ nhánh để thu hẹp không gian nghiệm. Tại mỗi nút ω , áp dụng lần lượt hai chiến lược sau.

Chiến lược 1: Rẽ nhánh theo số xe.

Tính tổng luồng rời depot trong nghiệm LP:

$$\bar{l} = \sum_{j \in I} \bar{x}_{0j}.$$

Nếu $\bar{l} \notin \mathbb{Z}$, tạo hai nút con:

$$\omega_L = \omega \cup \left\{ \sum_{j \in I} x_{0j} \leq \lfloor \bar{l} \rfloor \right\}, \quad \omega_R = \omega \cup \left\{ \sum_{j \in I} x_{0j} \geq \lceil \bar{l} \rceil \right\}.$$

Nhánh ω_L ép số xe không vượt quá $\lfloor \bar{l} \rfloor$, nhánh ω_R ép số xe không nhỏ hơn $\lceil \bar{l} \rceil$. Chiến lược này cố định quy mô đội xe trước khi xử lý các biến cung, giúp loại bỏ sớm các cấu hình số xe không khả thi.

Chiến lược 2: Rẽ nhánh theo biến cung.

Nếu $\bar{l} \in \mathbb{Z}$ nhưng vẫn tồn tại biến cung phân số, chọn:

$$\bar{x}_{uv} = \arg \min_{0 < \bar{x}_{ij} < 1} |\bar{x}_{ij} - 0.5|,$$

tức là biến phân số gần 0.5 nhất — biến có mức độ không chắc chắn cao nhất. Tạo hai nút con:

$$\omega_L = \omega \cup \{x_{uv} = 0\}, \quad \omega_R = \omega \cup \{x_{uv} = 1\}.$$

Nhánh ω_L loại cung (u, v) khỏi mọi tuyến, nhánh ω_R bắt buộc cung (u, v) xuất hiện trong nghiệm. Hai nhánh con phân hoạch không gian nghiệm thành hai tập rời nhau và đầy đủ, đảm bảo không bỏ sót nghiệm tối ưu.

Sau khi tạo ω_L và ω_R , mỗi nút con được kiểm tra nhanh tính khả thi sơ bộ về số xe, sức chứa và cửa sổ thời gian trước khi đẩy vào stack. Nút nào vi phạm bị loại ngay mà không cần giải LP.

Tóm lại, thuật toán Branch-and-Bound giải bài toán VRPTW bằng cách liên tục chia nhỏ không gian nghiệm dựa trên các biến nhị phân x_{ij} . Ở mỗi nút, nghiệm của bài toán nối lỏng tuyến tính cung cấp cận dưới, từ đó giúp loại bỏ các nhánh không thể dẫn đến nghiệm tốt hơn.

1.3 Pseudocode

Thuật toán 1: Branch-and-Bound (DFS) cho bài toán VRPTW [2]

Input: Mô hình MILP P_0 của VRPTW: tập khách hàng I , depot 0, chi phí c_{ij} , sức chứa Q , cửa sổ thời gian $[a_i, b_i]$, số xe tối đa m .

Output: Nghiệm tối ưu $S^* = (x^*, t^*, y^*)$, chi phí tối ưu Z^* .

```

1:  $\mathcal{S} \leftarrow [\omega_0], \quad Z^* \leftarrow +\infty, \quad S^* \leftarrow \emptyset$ 
2: while  $\mathcal{S} \neq \emptyset$  do
3:    $\omega \leftarrow \text{pop}(\mathcal{S})$ 
4:   Giải LP( $\omega$ ): thay  $x_{ij} \in \{0, 1\}$  bằng  $0 \leq x_{ij} \leq 1$ 
5:   if LP( $\omega$ ) vô nghiệm then continue
6:   end if
7:    $(\bar{x}, \bar{t}, \bar{y}) \leftarrow$  nghiệm tối ưu của LP( $\omega$ )
8:    $LB(\omega) \leftarrow \sum_{i \in I_0} \sum_{j \in I_0, j \neq i} c_{ij} \bar{x}_{ij}$ 
9:   if  $LB(\omega) \geq Z^*$  then continue
10:  end if
11:  if  $\bar{x}_{ij} \in \{0, 1\} \forall i, j$  then
12:    if  $\bar{x}$  tạo tuyến hợp lệ và  $LB(\omega) < Z^*$  then
13:       $Z^* \leftarrow LB(\omega), \quad S^* \leftarrow (\bar{x}, \bar{t}, \bar{y})$ 
14:    end if
15:    continue
16:  end if
17:   $\bar{l} \leftarrow \sum_{j \in I} \bar{x}_{0j}$ 
18:  if  $\bar{l} \notin \mathbb{Z}$  then
19:     $\omega_L \leftarrow \omega \cup \left\{ \sum_{j \in I} x_{0j} \leq \lfloor \bar{l} \rfloor \right\}, \quad \omega_R \leftarrow \omega \cup \left\{ \sum_{j \in I} x_{0j} \geq \lceil \bar{l} \rceil \right\}$ 
20:  else
21:     $\bar{x}_{uv} \leftarrow \arg \min_{0 < \bar{x}_{ij} < 1} |\bar{x}_{ij} - 0.5|$ 
22:     $\omega_L \leftarrow \omega \cup \{x_{uv} = 0\}, \quad \omega_R \leftarrow \omega \cup \{x_{uv} = 1\}$ 
23:  end if
24:  for  $\omega' \in \{\omega_L, \omega_R\}$  do
25:    if  $\omega'$  không vi phạm ràng buộc số xe, sức chứa, cửa sổ thời gian then

```

```

26:         push( $\mathcal{S}$ ,  $\omega'$ )
27:     end if
28: end for
29: end while
30: return  $S^*$ ,  $Z^*$ 

```

1.4 Độ phức tạp của thuật toán

Thuật toán Branch-and-Bound với DFS trong trường hợp xấu nhất phải duyệt toàn bộ cây tìm kiếm. Với n khách hàng, số biến nhị phân x_{ij} là $O(n^2)$, mỗi lần rẽ nhánh tạo hai nút con, do đó kích thước cây có thể lên tới:

$$|T| = O(2^{n^2}) \text{ nút.}$$

Chi phí tại mỗi nút. Theo pseudocode, mỗi nút ω được xử lý qua các bước sau.

- **Giải LP(ω).** Mô hình LP có $O(n^2)$ biến và $O(n^2)$ ràng buộc. Chi phí giải bằng simplex hoặc interior-point là $O(n^{4.5})$.
- **Cắt tia.** So sánh $LB(\omega)$ với Z^* : $O(1)$.
- **Kiểm tra nghiệm nguyên.** Duyệt toàn bộ $O(n^2)$ biến $\bar{x}_{ij}$ để kiểm tra tính nguyên, truy vết tuyến và xác nhận ràng buộc: $O(n^2)$.
- **Rẽ nhánh.** Tính $\bar{l} = \sum_{j \in I} \bar{x}_{0j}$ mất $O(n)$; chọn $\bar{x}_{uv} = \arg \min_{0 < \bar{x}_{ij} < 1} |\bar{x}_{ij} - 0.5|$ mất $O(n^2)$.
- **Lọc nút con.** Kiểm tra nhanh vi phạm số xe, sức chứa và cửa sổ thời gian trên hai nút ω_L, ω_R : $O(n)$.

Chi phí mỗi nút bị chi phối bởi bước giải LP:

$$T_{\text{node}} = O(n^{4.5}).$$

Độ phức tạp thời gian.

$$T_{\text{total}} = O(2^{n^2} \cdot n^{4.5}).$$

Trên thực tế, cắt tia theo $LB(\omega) \geq Z^*$ loại bỏ sớm phần lớn các nhánh, nên số nút thực sự được xét $N_{\text{visited}} \ll 2^{n^2}$.

Độ phức tạp không gian. Mỗi nút lưu tập ràng buộc bổ sung tích lũy từ gốc đến nút đó, tốn $O(n^2)$ bộ nhớ. Với DFS, stack $\mathcal{S}$ chứa tối đa $O(n^2)$ nút tại mỗi thời điểm (bằng độ sâu cây), do đó:

$$S_{\text{DFS}} = O(n^4).$$

2 BRANCH AND CUT

2.1 Khái quát

Branch-and-Cut là một phương pháp nâng cao của Branch-and-Bound, được sử dụng rộng rãi để giải các bài toán tối ưu tổ hợp khó như VRPTW. Phương pháp này kết hợp hai kỹ thuật: **Branch-and-Bound** (phân nhánh và cận) và **Cutting Planes** (thêm các mặt cắt).

Ý tưởng cốt lõi của Branch-and-Cut là không chỉ phân nhánh khi gặp nghiệm phân số mà còn chủ động tìm và thêm các ràng buộc cắt — các *bất đẳng thức hợp lệ* (valid inequalities) — vào mô hình LP relaxation tại từng nút của cây tìm kiếm. Những ràng buộc này thắt chặt vùng nghiệm khả thi, nâng cao chất lượng cận dưới (lower bound) một cách đáng kể, từ đó tăng hiệu quả cắt tỉa và giảm mạnh số lượng nút cần duyệt so với Branch-and-Bound thuần túy.

2.2 Bất đẳng thức hợp lệ (Valid Inequalities)

Ta sẽ thêm vào tập ràng buộc của bài toán LP Relaxation các bất đẳng thức hợp lệ (Valid Inequalities) là các ràng buộc/ lát cắt nhằm thu nhỏ không gian nghiệm bằng cách loại bỏ một phần nghiệm không nguyên.

2.2.1 Capacity Inequalities

Xét bài toán trên đồ thị có hướng $G = (V, A)$ với tập đỉnh $V = N \cup \{0\}$ (N là tập khách hàng và 0 là kho depot). Xét một tập con khách hàng $S \subseteq N$. Bất đẳng thức sức chứa (Capacity Inequalities) đóng vai trò triệt tiêu các chu trình con (subtour), đồng thời đảm bảo số lượng xe rời khỏi tập S không nhỏ hơn số xe tối thiểu cần thiết để phục vụ toàn bộ nhu cầu của S .

Bất đẳng thức hợp lệ được thêm trực tiếp vào mô hình có dạng:

$$\sum_{(i,j) \in A: i \in S, j \in V \setminus S} x_{ij} \geq k(S) \quad (1)$$

trong đó $k(S) = \left\lceil \frac{\sum_{i \in S} q_i}{Q} \right\rceil$ là số lượng xe tối thiểu cần thiết để chở hết tổng lượng hàng hóa của tập khách hàng S với sức chứa tối đa của mỗi xe là Q .

Hàm 1: Kiểm tra và thêm ràng buộc sức chứa nếu vi phạm

```

1: function ADD_CAPACITY_CONSTRAINT(LP( $\omega$ ),  $S, \hat{x}$ )
2:   sum  $\leftarrow$  0
3:   for  $i \in S$  do
4:     sum  $\leftarrow$  sum +  $q_i$                                 ▷ Tính tổng nhu cầu của tập siêu nút
5:   end for
6:    $k(S) \leftarrow \lceil \text{sum}/Q \rceil$                                 ▷ Số xe tối thiểu theo lý thuyết
7:    $f(S) \leftarrow \sum_{(i,j) \in A: i \in S, j \in V \setminus S} \hat{x}_{ij}$     ▷ Số xe trên nghiệm  $\hat{x}$  hiện tại
8:   if  $f(S) < k(S)$  then
9:     Ràng buộc mới:  $C \leftarrow (\sum_{(i,j) \in A: i \in S, j \in V \setminus S} x_{ij} \geq k(S))$ 
10:    LP( $\omega$ )  $\leftarrow$  LP( $\omega$ )  $\cup \{C\}$ 
11:   end if
12: end function

```

Đồng thời, ràng buộc sức chứa này cũng đóng vai trò là một bất đẳng thức loại bỏ chu trình con (Subtour Elimination Constraint) theo Jonathan F. Bard [1].

Ví dụ: Giả sử đội xe có sức chứa tối đa $Q = 20$. Xét tập khách hàng $S = \{1, 2, 3\}$ với nhu cầu hàng hóa tương ứng là $q_1 = 8$, $q_2 = 7$, và $q_3 = 10$. Tổng nhu cầu của toàn tập S là $\sum_{i \in S} q_i = 25$.

Vì tổng nhu cầu (25) vượt quá sức chứa của một xe (20), số lượng xe tối thiểu bắt buộc phải tham gia phục vụ tập khách hàng này là $k(S) = \lceil 25/20 \rceil = 2$ xe. Lúc này, bất đẳng thức sức chứa (1) được thiết lập như sau:

$$\sum_{i \in \{1,2,3\}, j \in V \setminus \{1,2,3\}} x_{ij} \geq 2$$

2.2.2 2-Cycle Elimination Inequalities

Bên cạnh các nhất cắt động được thêm vào thông qua cơ chế Callback trong quá trình giải, một số chu trình con kích thước nhỏ có thể được nhận diện và triệt tiêu ngay từ giai đoạn

tiền xử lý. Cụ thể, chu trình con khép kín giữa hai khách hàng bất kỳ i và j ($i \rightarrow j \rightarrow i$) là hoàn toàn bất khả thi trong một lời giải nguyên hợp lệ của bài toán VRPTW.

Do đó, ta có thể chủ động thêm các ràng buộc **2-cycle elimination** nhằm loại bỏ chu trình hai nút vào mô hình ngay từ lúc khởi tạo với độ phức tạp thuật toán là $O(n^2)$. Các ràng buộc này có dạng tĩnh như sau:

$$x_{ij} + x_{ji} \leq 1, \quad \forall i, j \in N, i \neq j \quad (2)$$

Việc tích hợp sớm các ràng buộc tĩnh này giúp hệ thống thu hẹp đáng kể không gian tìm kiếm phân số ngay tại nút gốc.

2.3 Thuật toán co cụm đồ thị (Graph Shrinking)

Để tìm và xác định các tập khách hàng S vi phạm ràng buộc sức chứa (1) từ nghiệm phân số $\hat{x}$, nghiên cứu này áp dụng thuật toán Heuristic co cụm đồ thị (Graph Shrinking).

Ý tưởng cốt lõi là liên tục gộp các cặp nút khách hàng có luồng phân số $\hat{x}_{ij}$ lớn lại với nhau để tạo thành một siêu nút (supernode). Việc co cụm các liên kết mạnh nội bộ này làm cho tổng luồng đi ra khỏi siêu nút mới — ký hiệu là $f(S)$ — có xu hướng thu nhỏ lại, từ đó giúp thuật toán dễ dàng phát hiện ra các vùng có luồng thoát nhỏ hơn số xe tối thiểu cần thiết ($f(S) < k(S)$) để tiến hành lập nhất cắt. Cấu trúc dữ liệu Union-Find (Disjoint Set Union) được áp dụng để triển khai thuật toán trên.

Hàm 2: Heuristic Graph Shrinking

```

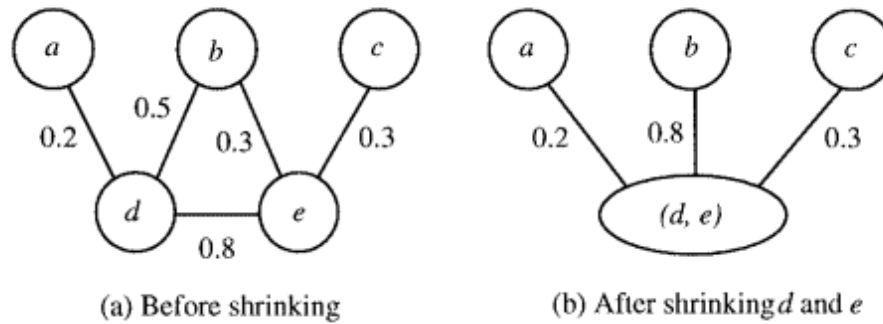
1: function GRAPH_SHRINKING( $\omega, \hat{x}, \mu, Q$ )
2:    $A_0 \leftarrow \{(i, j) \in A \mid i \neq 0 \text{ và } j \neq 0 \text{ và } \hat{x}_{ij} > \varepsilon\}$ 
3:   for iter = 1 to MAX_ITER do
4:     for each  $v \in N$  do
5:       MAKESET( $v$ )                                ▷ Khởi tạo:  $\text{parent}[v] \leftarrow v, q(v) \leftarrow q_v, S_v \leftarrow \{v\}$ 
6:     end for
7:      $L \leftarrow \text{sort}(A_0 \text{ theo } \hat{x}_{ij} \text{ giảm dần})$ 
8:     while  $L \neq \emptyset$  do
9:       Chọn ngẫu nhiên cung  $(u, v)$  trong  $\mu$  cung đầu tiên của  $L$ 
10:       $L \leftarrow L \setminus \{(u, v)\}$ 
```

```

11:       $r_u \leftarrow \text{FIND}(u)$ 
12:       $r_v \leftarrow \text{FIND}(v)$ 
13:      if  $r_u \neq r_v$  then                                ▷ Tương đương  $S_u \cap S_v = \emptyset$ 
14:           $\text{UNION}(r_u, r_v)$                                 ▷ Gộp hai tập hợp
15:           $r_{\text{new}} \leftarrow \text{FIND}(u)$ 
16:           $S_{\text{new}} \leftarrow$  Tập hợp các nút thuộc nhánh  $r_{\text{new}}$ 
17:           $\text{ADD\_CAPACITY\_CONSTRAINT}(\text{LP}(\omega), S_{\text{new}}, \hat{x})$ 
18:      end if
19:  end while
20:  end for
21: end function

```

Tham số μ kiểm soát mức độ ngẫu nhiên trong quá trình chọn cung co cụm. Trong thực nghiệm, μ được đặt bằng 5, tức mỗi lần co cụm chọn ngẫu nhiên một trong 5 cung có $\hat{x}_{ij}$ lớn nhất còn lại trong L .



Hình 1: Minh họa quá trình co cụm đồ thị

Đánh giá độ phức tạp. Độ phức tạp của thuật toán Graph Shrinking dựa trên cấu trúc dữ liệu Union-Find được quyết định bởi hai giai đoạn chính trong mỗi vòng lặp MAX_ITER. Đầu tiên, thuật toán thực hiện sắp xếp danh sách các cung L , tiêu tốn $O(|A| \log |A|)$. Sau đó thuật toán duyệt qua danh sách các cung đã sắp xếp để thực hiện gộp siêu nút. Nhờ áp dụng cấu trúc Union-Find, mỗi thao tác tìm kiếm (Find) và hợp nhất (Union) có chi phí trung bình là $O(\alpha(n))$, trong đó α là hàm ngược Ackermann (xấp xỉ hằng số). Do đó, toàn bộ vòng lặp while tốn khoảng $O(|A|)$ thời gian.

Tổng hợp lại, độ phức tạp tính toán cho một lượt lặp là $O(|A| \log |A|)$. Sau MAX_ITER lần lặp, độ phức tạp tổng quát của thuật toán là:

$$O(\text{MAX_ITER} \cdot |A| \log |A|)$$

Với đồ thị thưa $|A| = O(n)$, thuật toán có độ phức tạp $O(n \log n)$. Trong trường hợp đồ thị dày $|A| = O(n^2)$, độ phức tạp là $O(n^2 \log n)$.

2.4 Sơ đồ thuật toán Branch-and-Cut

Branch-and-Cut giữ nguyên toàn bộ khung duyệt cây của Branch-and-Bound. Điểm khác biệt duy nhất là sau khi giải LP relaxation tại mỗi nút, ta chèn thêm một **vòng lặp tìm mặt cắt** (cutting plane loop) trước khi quyết định phân nhánh. Cụ thể, vòng lặp này:

1. Chạy thuật toán co cụm đồ thị để tìm các bất đẳng thức (1) bị vi phạm.
2. **Nếu tìm thấy mặt cắt:** Thêm các ràng buộc vi phạm vào mô hình LP và giải lại — lặp lại cho đến khi không còn vi phạm nào được phát hiện.
3. **Nếu không còn mặt cắt:** Thoát khỏi vòng lặp và tiến hành phân nhánh hoặc cập nhật nghiệm như Branch-and-Bound thông thường.

Nhờ cơ chế này, cận dưới tại mỗi nút được thắt chặt tối đa trước khi phân nhánh, giúp cắt tĩa cây tìm kiếm hiệu quả hơn đáng kể. Ngoài ra vòng lặp được thiết kế với cơ chế ngắt sớm (early stopping) nhằm tối ưu chi phí tính toán. Cụ thể, vòng lặp sẽ dừng lại nếu thỏa mãn một trong ba điều kiện: (1) thuật toán Heuristic không phát hiện thêm nhất cắt vi phạm nào mới; (2) số vòng lặp đạt đến giới hạn MAX_ITER(ω), trong đó giới hạn này thường được nới lỏng ở nút gốc nhưng thắt chặt ở các nút sâu để tránh lãng phí thời gian; hoặc (3) xuất hiện hiện tượng *tail-off*, nghĩa là các mặt cắt mới thêm vào quá yếu, khiến cận dưới cải thiện không đáng kể ($\Delta LB \leq \varepsilon$, với ε là một dung sai rất nhỏ).

Vòng lặp Cutting Plane

1: // Chèn vào sau bước giải LP Relaxation của Branch-and-Bound ở dòng 10

2: ADD_STATIC_2_CYCLE_CUTS(ω)

▷ Thêm ràng buộc $x_{ij} + x_{ji} \leq 1$

```

3: iter  $\leftarrow$  0
4:  $LB_{\text{prev}} \leftarrow -\infty$ 
5: repeat // Vòng lặp tìm mặt cắt (Cutting Plane Loop)
6:   iter  $\leftarrow$  iter + 1
7:   GRAPH_SHRINKING( $\omega, \bar{x}, \mu$ ) // Tự động quét và nạp các mặt cắt vi phạm
8:   Giải lại LP( $\omega$ )
9:   if LP( $\omega$ ) vô nghiệm then break
10:  end if
11:  ( $\bar{x}, \bar{t}, \bar{y}$ )  $\leftarrow$  nghiệm tối ưu của LP( $\omega$ )
12:   $LB(\omega) \leftarrow \sum_i \sum_j c_{ij} \bar{x}_{ij}$ 
13:  if  $LB(\omega) \geq Z^*$  then break
14:  end if
15:   $\Delta LB \leftarrow LB(\omega) - LB_{\text{prev}}$ 
16:   $LB_{\text{prev}} \leftarrow LB(\omega)$ 
17: until Không có nhất cắt mới hoặc iter  $\geq$  MAX_ITER( $\omega$ ) hoặc  $\Delta LB \leq \varepsilon$ 
18:
19: // Tiếp tục bước kiểm tra tính nguyên hoặc phân nhánh ở dòng 11

```

2.4.1 Đánh giá độ phức tạp

Mặc dù kích thước cây trong kịch bản xấu nhất của Branch-and-Cut (B&C) vẫn là $O(2^{n^2})$, chi phí tính toán tại từng nút và tổng thời gian thực thi có sự khác biệt rõ rệt so với Branch-and-Bound (B&B) thuần túy.

Độ phức tạp thời gian. Tại từng nút, B&C phải trả thêm chi phí phụ để chạy heuristic tìm nhất cắt $O(n^2 \log n)$ và giải lại LP liên tục $p(\omega)$ lần, mỗi lần tốn $O(n^{4.5})$. Do đó, chi phí tại một nút là:

$$T_{\text{node}}^{\text{B\&C}} = O(p(\omega) \cdot n^{4.5})$$

Vậy độ phức tạp tổng thời gian của B&C được tính bằng:

$$T_{\text{total}}^{\text{B\&C}} = O(N_{\text{visited}} \cdot \bar{p} \cdot n^{4.5})$$

với $\bar{p}$ là số lần giải lại LP trung bình. Mặc dù việc tìm nhất cắt và giải lại LP khiến B&C tốn thêm chi phí tính toán tại mỗi nút, nhưng trên thực tế tốc độ thực thi tổng thể của B&C lại

nhANH hơn B&B đáng kể. Điều này là do các nhát cắt giúp thu hẹp không gian nghiệm nhanh hơn, làm giảm số lượng nút cần duyệt (N_{visited}).

Độ phức tạp không gian. Tương tự như Branch-and-Bound, với DFS chiều sâu tối đa của cây tìm kiếm vẫn là $O(n^2)$ do ta phân nhánh trên các biến x_{ij} . Tuy nhiên tại mỗi nút ω , hệ thống còn phải lưu thêm các nhát cắt (Capacity Cuts) được nạp vào. Gọi C_{max} là số lượng mặt cắt tối đa được duy trì trong mô hình $LP(\omega)$ tại một nút. Bộ nhớ cấp phát cho mỗi nút lúc này là $O(n^2 + C_{\text{max}})$.

Bộ giải còn cần duy trì thêm một Kho mặt cắt chung (Global Cut Pool) để tái sử dụng các bất đẳng thức hợp lệ cho nhiều nhánh khác nhau trên cây, tiêu tốn một vùng nhớ $O(P_{\text{pool}})$.

Do đó, độ phức tạp không gian tổng quát của Branch-and-Cut là:

$$S_{\text{B\&C}} = O(n^2 \cdot (n^2 + C_{\text{max}}) + P_{\text{pool}}) = O(n^4 + n^2 C_{\text{max}} + P_{\text{pool}}).$$

Dù tồn thêm bộ nhớ lưu trữ kho mặt cắt (cut pool), B&C lại giảm đáng kể lượng nút trong hàng đợi nhờ khả năng cắt tỉa mạnh. Điều này giúp tiết kiệm tài nguyên RAM tổng thể, đảm bảo hệ thống vận hành ổn định hơn.

3 BRANCH AND PRICE

3.1 Khái quát

Branch and Price (Nhánh và Giá) là một phương pháp giải chính xác kết hợp hai kỹ thuật: *Branch and Bound* và *Column Generation*. Thay vì liệt kê toàn bộ các tuyến đường khả thi ngay từ đầu – một điều không khả thi vì số lượng tuyến đường tăng theo cấp số mũ với số khách hàng – thuật toán chỉ duy trì một tập con nhỏ các tuyến đường trong *Restricted Master Problem* (RMP), và bổ sung dần các tuyến đường có tiềm năng cải thiện nghiệm thông qua bài toán con SPPRC.

Trong cài đặt này, bài toán được giải bằng Python với thư viện gurobipy để giải RMP và thư viện sortedcontainers để quản lý tập nhãn trong SPPRC. Toàn bộ tham số bài toán (ma trận khoảng cách, time window, demand, ...) được quản lý qua lớp ParamsVRP, các tuyến đường được biểu diễn qua lớp Route.

3.2 Khởi tạo

Trước khi bắt đầu, toàn bộ tham số bài toán được nạp từ file dữ liệu Solomon thông qua `ParamsVRP.init_params()`. Trong cài đặt này, depot được nhân đôi: đỉnh 0 là depot xuất phát, đỉnh $N - 1$ là depot kết thúc (cùng tọa độ), để thuận tiện cho SPPRC tìm đường từ 0 đến $N - 1$. Vì vậy, tổng số đỉnh trong ma trận là $n + 2$ (depot đầu, n khách hàng, depot cuối).

Để ngăn các chu trình không hợp lệ, ma trận khoảng cách `dist_base` được khởi tạo với:

- $\text{dist}[i][0] = \text{verybig} \forall i$: cấm quay về depot đầu.
- $\text{dist}[N - 1][j] = \text{verybig} \forall j$: cấm rời depot cuối.
- $\text{dist}[i][i] = \text{verybig} \forall i$: cấm tự vòng.

Các tuyến đường khởi tạo được xây dựng theo nguyên tắc mỗi khách hàng một xe (dummy routes), đảm bảo RMP luôn có nghiệm khả thi ban đầu.

Thuật toán: Khởi tạo tuyến đường ban đầu

Input: Ma trận `dist`, số khách hàng n , depot cuối $N - 1$

Output: Tập tuyến khởi tạo R_0

- 1: $R_0 \leftarrow \emptyset$
 - 2: **for** mỗi khách hàng $i \in \{1, \dots, n\}$ **do**
 - 3: $c_r \leftarrow \text{dist}[0][i] + \text{dist}[i][N - 1]$
 - 4: Tạo tuyến r với đường đi $[0 \rightarrow i \rightarrow N - 1]$, chi phí c_r , mức sử dụng $Q_r = 1$
 - 5: $R_0 \leftarrow R_0 \cup \{r\}$
 - 6: **end for**
 - 7: **return** R_0
-

Với bộ dữ liệu R101 (100 khách hàng), điều này tạo ra 100 tuyến đường khởi tạo, mỗi tuyến phục vụ đúng một khách hàng.

3.3 Column Generation

3.3.1 Restricted Master Problem (RMP)

RMP là bài toán Set Covering được giải bằng Gurobi ở dạng liên tục (LP Relaxation).
Mô hình:

$$\begin{aligned} \min \quad & \sum_{r \in R} c_r y_r \\ \text{s.t.} \quad & \sum_{r \in R} a_{ir} y_r \geq 1 \quad \forall i \in \{1, \dots, N-2\} \\ & y_r \geq 0 \quad \forall r \in R \end{aligned}$$

Trong đó:

- R : tập tuyến đường hiện có trong RMP.
- $c_r = \sum_{(i,j) \in r} \text{dist}[i][j]$: tổng khoảng cách Euclid của tuyến r , tính từ ma trận `dist_base`.
- $a_{ir} = 1$ nếu khách hàng i nằm trong tuyến r (tức $i \in \text{route.path}[1 : -1]$).
- $y_r \geq 0$: mức sử dụng tuyến r (thực, được nói lỏng từ nhị phân để giải LP).

Ràng buộc phủ khách hàng được lập cho tất cả đỉnh từ 1 đến $N-2$ (bỏ qua depot đầu 0 và depot cuối $N-1$). Các cột mới được bổ sung *tăng dần* (incremental) vào mô hình mà không tái tạo lại từ đầu, nhờ đó giảm chi phí tính toán.

Thuật toán: Thêm cột mới vào RMP

Input: Tuyến mới r , tập ràng buộc phủ $\{\text{constr}_i\}_{i=1}^{N-2}$, ma trận `dist`

Output: Biến y_r được thêm vào RMP

- 1: $c_r \leftarrow \sum_{(i,j) \in r} \text{dist}[i][j]$
 - 2: Tạo cột col với hệ số 1 tại mọi ràng buộc constr_i mà khách hàng $i \in r$
 - 3: Thêm biến $y_r \geq 0$ vào RMP với hệ số mục tiêu c_r và cột col
-

Sau khi hội tụ, giá trị y_r của mỗi tuyến được lưu vào `route.Q` – đây là giá trị thực phản ánh mức độ tuyến đó được sử dụng trong nghiệm LP. Giá trị Q này được dùng trong B&B để kiểm tra tính nguyên và tính toán ma trận `edges`.

3.3.2 Bài toán con: Cập nhật reduced cost và gọi SPPRC

Sau mỗi lần giải RMP, thuật toán lấy giá trị đối ngẫu $\pi_i = \text{constraints}[i].\pi_i$ của từng ràng buộc khách hàng i , rồi cập nhật ma trận chi phí cho SPPRC:

$$\text{cost}[i][j] = \text{dist}[i][j] - \pi_i \quad \forall i \in \{1, \dots, N-2\}, j \in V$$

Lưu ý: hàng 0 (depot đầu) và cột $N-1$ (depot cuối) của cost **không bị trừ** π_i vì depot không có ràng buộc phủ. Điều kiện sinh cột mới: SPPRC tìm được tuyến đường có reduced cost âm, tức:

$$\bar{c}_r = \sum_{(i,j) \in r} \text{cost}[i][j] < 0$$

Nếu không còn tuyến nào như vậy, Column Generation kết thúc – RMP đã đạt tối ưu LP. Vòng lặp Column Generation được thực hiện trong `ColumnGeneration.compute_col_gen()`:

Thuật toán: Column Generation

Input: RMP với tập tuyến R_0 , ma trận dist , ttime

Output: Giá trị tối ưu Z_{LP} và tập tuyến R

- 1: Khởi tạo RMP với các tuyến trong R_0
- 2: Thêm biến $y_r \geq 0, \forall r \in R_0$
- 3: Thêm ràng buộc phủ khách hàng:

$$\sum_{r \in R} a_{ir} y_r \geq 1, \quad \forall i = 1, \dots, N-2$$

- 4: **while** true **do**
- 5: Giải RMP $\rightarrow Z_{LP}$
- 6: **if** RMP không có nghiệm tối ưu **then**
- 7: **break**
- 8: **end if**
- 9: Lấy dual variables π_i từ các ràng buộc phủ
- 10: Cập nhật reduced cost: $\text{cost}[i][j] \leftarrow \text{dist}[i][j] - \pi_i$
- 11: Gọi SPPRC tìm tuyến mới có $\bar{c}_r < 0$
- 12: **if** không tìm được tuyến mới **then**

```

13:      break                                     ▷ RMP đã tối ưu LP
14:  end if
15:  for mỗi tuyến  $r$  do
16:       $c_r \leftarrow \sum_{(i,j) \in r} \text{dist}[i][j]$ 
17:      Tạo cột  $col$  từ các khách hàng mà  $r$  phủ
18:      Thêm biến  $y_r$  vào RMP với  $c_r$  và  $col$ 
19:  end for
20: end while
21: Gán  $\text{route.Q} \leftarrow y_r$  cho mỗi tuyến  $r$ 
22: return  $Z_{LP}, R$ 

```

3.4 Bài toán con SPPRC

Khái quát

SPPRC (Shortest Path Problem with Resource Constraints) tìm tuyến đường từ depot 0 đến depot cuối $N - 1$ có reduced cost âm nhỏ nhất, thỏa mãn đồng thời ba ràng buộc tài nguyên đặc trưng của VRPTW:

- **Sức chứa:** $\sum_{i \in r} d[i] \leq Q$.
- **Khung thời gian:** thời gian đến khách hàng i phải thỏa $\leq b[i]$; nếu đến sớm hơn $a[i]$ thì xe chờ.
- **Không lặp đỉnh:** mỗi khách hàng xuất hiện tối đa một lần trong tuyến.

Thuật toán sử dụng kỹ thuật *Label Setting* theo Irnich & Desaulniers. Mỗi nhãn (label) L biểu diễn một đường đi khả thi từ depot đến nút hiện tại:

$$L = (\text{city}, \text{cost}, \text{ttime}, \text{demand}, \text{vertex_visited})$$

Trong đó:

- city : đỉnh hiện tại của nhãn.
- cost : tổng reduced cost tích lũy theo ma trận $\text{cost}[][]$.

- $ttime$: thời gian thực tế đến $city$, tính cả thời gian chờ (nếu đến trước a_i) và thời gian phục vụ s tại đỉnh trước.
- $demand$: tổng nhu cầu tích lũy theo ma trận $d[]$.
- $vertex_visited$: mảng boolean $n + 2$ phần tử, đánh dấu các đỉnh đã thăm.

Hàm mở rộng nhãn (REF)

Từ nhãn L tại đỉnh u , mở rộng sang đỉnh v nếu $dist[u][v] < verybig$ và v chưa được thăm:

$$ttime' = \max(a[v], ttime + ttime[u][v] + s[u])$$

$$demand' = demand + d[v]$$

$$cost' = cost + cost[u][v]$$

Nhãn mới được chấp nhận khi $ttime' \leq b[v]$ và $demand' \leq Q$. Lưu ý: $ttime[u][v]$ được tính từ $dist_base[u][v]$ chia cho $speed$ (mặc định = 1), còn $cost[u][v]$ là reduced cost đã trừ π_u .

Kỹ thuật tăng tốc (Feillet 2004)

Ngay sau khi mở rộng đến đỉnh v , thuật toán kiểm tra trước từng khách hàng j chưa thăm. Nếu từ v đi thẳng đến j đã vi phạm time window hoặc capacity thì đánh dấu j như đã thăm, khóa trước để tránh mở rộng vô ích:

Thuật toán: Feillet 2004 – Khóa trước đỉnh không thể thăm

Input: Nhãn mới tại đỉnh v với thời gian tt' , tải d' ; ma trận $ttime$, d , b ; sức chứa Q

Output: Mảng $visited'$ được cập nhật

- 1: **for** mỗi khách hàng $j \in \{1, \dots, N - 2\}$ chưa thăm **do**
- 2: **if** $tt' + ttime[v][j] + s[v] > b[j]$ **hoặc** $d' + d[j] > Q$ **then**
- 3: $visited'[j] \leftarrow \text{True}$ ▷ Khóa trước: không thể thăm j từ v
- 4: **end if**
- 5: **end for**

Kỹ thuật này cắt giảm đáng kể số nhãn cần xử lý, đặc biệt hiệu quả với các bộ dữ liệu có time window hẹp như C101.

Kiểm tra dominance

Nhãn L_1 *dominate* nhãn L_2 tại cùng đỉnh nếu L_1 không tệ hơn L_2 trên mọi tài nguyên và tập đỉnh đã thăm:

$$L_1.\text{cost} \leq L_2.\text{cost} \wedge L_1.\text{ttime} \leq L_2.\text{ttime} \wedge L_1.\text{demand} \leq L_2.\text{demand}$$

$$\wedge \forall k: L_1.\text{visited}[k] = \text{True} \Rightarrow L_2.\text{visited}[k] = \text{True}$$

Điều kiện tập đỉnh đảm bảo rằng L_1 có ít nhất mọi sự linh hoạt mở rộng mà L_2 có (nếu L_2 đã thăm k thì L_1 cũng phải đã thăm – tức L_1 không bị hạn chế hơn L_2). Nhãn bị dominate bị xóa khỏi tập chưa xử lý U , thu hẹp không gian tìm kiếm. Trong cài đặt, điều kiện này được kiểm tra theo cặp trong danh sách nhãn tại mỗi đỉnh.

Thu thập nghiệm và truy vết đường đi

Các nhãn có $\text{cost} < -10^{-4}$ đến được depot cuối $N - 1$ được đưa vào tập P . Tuyến đường được tái tạo bằng cách truy vết ngược qua chuỗi con trở nhãn trước, sau đó đảo ngược để có thứ tự depot đầu $\rightarrow \dots \rightarrow$ depot cuối. Thuật toán trả về tối đa $\text{maxsol} = 2 \times (N - 1)$ tuyến có reduced cost âm để đưa vào RMP.

Thuật toán: SPPRC – Label Setting

Input: Đồ thị G , ma trận cost , dist , ttime , a , b , d , s , Q , maxsol

Output: Danh sách các tuyến có reduced cost âm

- 1: Tạo nhãn gốc $L_0 = (0, \text{cost} = 0, \text{ttime} = 0, \text{demand} = 0, \text{visited} = \{0\})$
- 2: $U \leftarrow \{L_0\}$, $P \leftarrow \emptyset$, $\text{nbsol} \leftarrow 0$
- 3: **while** $U \neq \emptyset$ và $\text{nbsol} < \text{maxsol}$ **do**
- 4: $L \leftarrow$ lấy nhãn có cost nhỏ nhất từ U
- 5: Kiểm tra dominance: với mọi L' trong danh sách nhãn tại $L.\text{city}$
- 6: **nếu** L dominate L' : đánh dấu $L'.\text{dominated} = \text{True}$, xóa L' khỏi U

```

7:      nếu  $L'$  dominate  $L$ : đánh dấu  $L.\text{dominated} = \text{True}$ , break
8:      if  $L.\text{dominated}$  then
9:          continue
10:     end if
11:     if  $L.\text{city} = N - 1$  then
12:         if  $L.\text{cost} < 0$  then
13:             Thêm  $L$  vào  $P$ ; cập nhật  $\text{nbsol}$ 
14:         end if
15:     else đỉnh  $v \in V$  chưa thăm,  $\text{dist}[L.\text{city}][v] < \text{verybig}$ 
16:          $tt' \leftarrow \max(a[v], L.\text{ttime} + \text{ttime}[L.\text{city}][v] + s[L.\text{city}])$ 
17:          $d' \leftarrow L.\text{demand} + d[v]$ 
18:         if  $tt' \leq b[v]$  và  $d' \leq Q$  then
19:             Sao chép  $\text{visited}'$  từ  $L.\text{visited}$ , đặt  $\text{visited}'[v] \leftarrow \text{True}$ 
20:              $\triangleright$  Feillet 2004: khóa trước các đỉnh không thể thăm tiếp  $j \in \{1, \dots, N - 2\}$ 
                chưa thăm
21:             if  $tt' + \text{ttime}[v][j] + s[v] > b[j]$  hoặc  $d' + d[j] > Q$  then
22:                  $\text{visited}'[j] \leftarrow \text{True}$ 
23:             end if
24:
25:             Tạo nhãn mới  $L' = (v, L.\text{cost} + \text{cost}[L.\text{city}][v], tt', d', \text{visited}')$ 
26:             Thêm  $L'$  vào  $U$  và danh sách nhãn tại  $v$ 
27:         end if
28:
29:     end if
30: end while
        nhãn  $L \in P$  không bị dominate,  $L.\text{cost} < -10^{-4}$ 
31: Truy vết ngược qua con trỏ nhãn trước để tái tạo tuyến đường
32: Đảo ngược thứ tự để có dạng depot đầu  $\rightarrow \dots \rightarrow$  depot cuối
33: Thêm tuyến vào danh sách kết quả
34:
35: return danh sách tuyến đường

```

3.5 Branch and Bound trong Branch and Price

Cấu trúc cây và thông tin tại mỗi nút

Cây B&B được biểu diễn qua lớp TreeBB với các trường:

- father: con trỏ đến nút cha.
- son0: con trỏ đến nút con đầu tiên (nhánh 1 được xử lý trước).
- branch_from, branch_to: cạnh (i^*, j^*) được chọn để phân nhánh tại nút này.
- branch_value: 0 nếu cấm cạnh (i^*, j^*) , 1 nếu bắt buộc cạnh (i^*, j^*) .
- lowest_value: giá trị LP tối ưu tại nút này – đóng vai trò cận dưới cục bộ.
- toplevel: nhãn đánh dấu nút cấp cao nhất hiện tại, dùng để cập nhật LB toàn cục.

Chọn cạnh phân nhánh

Sau khi Column Generation hội tụ, thuật toán tính ma trận edges:

$$\text{edges}[i][j] = \sum_{r \in R} \text{route}.\text{Q}[r] \cdot \mathbf{1}[(i, j) \in r]$$

Nếu tồn tại cạnh (i, j) có $0 < \text{edges}[i][j] < 1$ (giá trị phân số), nghiệm chưa nguyên và cần phân nhánh. Tiêu chí chọn cạnh phân nhánh (i^*, j^*) :

$$(i^*, j^*) = \arg \max_{(i,j): 0 < \text{edges}[i][j] < 1} \min(\text{edges}[i][j], |1 - \text{edges}[i][j]|) \times \text{routes}[i].\text{cost}$$

Tiêu chí này ưu tiên cạnh vừa xa 0 và 1 nhất, vừa thuộc tuyến có chi phí lớn – tức cạnh mà việc phân nhánh sẽ tạo ra sự thay đổi lớn nhất trong cây tìm kiếm. Giá trị `best_val` được đặt là 0 nếu $|1 - \text{coef}| > \text{coef}$ (cạnh gần 0 hơn), ngược lại là 1.

Áp đặt ràng buộc phân nhánh lên ma trận dist

Thay vì thêm ràng buộc vào mô hình LP, cài đặt này thực hiện phân nhánh trực tiếp lên ma trận `dist`. Điều này tự động loại bỏ các tuyến vi phạm khỏi SPPRC vì SPPRC chỉ xét cạnh có $\text{dist}[u][v] < \text{verybig}$:

Thuật toán: Áp đặt ràng buộc phân nhánh lên dist

Input: Nút phân nhánh với cạnh (i^*, j^*) , giá trị branch_value , ma trận dist , dist_base

```

1: if  $\text{branch\_value} = 0$  then                                ▷ Cấm cạnh  $(i^*, j^*)$ 
2:    $\text{dist}[i^*][j^*] \leftarrow \text{verybig}$ 
3: else                                                        ▷ Bắt buộc cạnh  $(i^*, j^*)$ 
4:   if  $i^* \neq 0$  then                                         ▷ Không áp dụng nếu  $i^*$  là depot đầu
5:     Cấm toàn bộ cạnh ra khỏi  $i^*$ :  $\text{dist}[i^*][k] \leftarrow \text{verybig}, \forall k$ 
6:     Khôi phục cạnh bắt buộc:  $\text{dist}[i^*][j^*] \leftarrow \text{dist\_base}[i^*][j^*]$ 
7:   end if
8:   if  $j^* \neq N - 1$  then                                     ▷ Không áp dụng nếu  $j^*$  là depot cuối
9:     Cấm toàn bộ cạnh vào  $j^*$ :  $\text{dist}[k][j^*] \leftarrow \text{verybig}, \forall k$ 
10:    Khôi phục cạnh bắt buộc:  $\text{dist}[i^*][j^*] \leftarrow \text{dist\_base}[i^*][j^*]$ 
11:   end if
12:   Cấm cạnh ngược:  $\text{dist}[j^*][i^*] \leftarrow \text{verybig}$ 
13: end if

```

Nhờ cơ chế đệ quy khi xử lý nhánh thứ hai, toàn bộ chuỗi ràng buộc từ gốc đến nút hiện tại được tái áp dụng lên dist sau khi đã phục hồi dist_base .

Thuật toán: Phục hồi và tái áp dụng ràng buộc (nhánh 2)

```

1: Phục hồi  $\text{dist} \leftarrow \text{dist\_base}$ 
2: Áp dụng đệ quy toàn bộ chuỗi ràng buộc từ gốc đến nút hiện tại lên  $\text{dist}$ 

```

Lọc tuyến đường theo ràng buộc phân nhánh

Trước khi gọi đệ quy vào nút con, các tuyến đường không tương thích với ràng buộc phân nhánh bị loại:

- **Nhánh 1:** loại các tuyến có chứa cạnh (i^*, j^*) .
 - **Nhánh 2:** loại các tuyến có bất kỳ cạnh nào bị cấm trong dist hiện tại.
-

Thuật toán: Loại tuyến đường theo ràng buộc phân nhánh**Input:** Tập tuyến R , cạnh phân nhánh (i^*, j^*) , ma trận dist

- 1: ▷ Loại cho nhánh 1: loại tuyến chứa cạnh (i^*, j^*)
- 2: $R_1 \leftarrow \{r \in R \mid (i^*, j^*) \notin r\}$
- 3: ▷ Loại cho nhánh 2: loại tuyến có cạnh bị cấm
- 4: $R_2 \leftarrow \{r \in R \mid \forall (u, v) \in r : \text{dist}[u][v] < \text{verybig}\}$
- 5: **return** R_1, R_2

Hàm đánh giá cận (Bounding Function)

Theo lý thuyết nhánh cận, tại mỗi nút của cây tìm kiếm ứng với lời giải bộ phận – một tập ràng buộc phân nhánh đã áp đặt – cần xác định một hàm đánh giá cận g thỏa mãn:

$$g \leq \min \{f(x) : x \text{ là nghiệm nguyên mở rộng từ nút hiện tại}\}$$

tức là g là cận dưới của mọi nghiệm nguyên có thể phát sinh từ nút này. Nếu $g > f^*$ (cận trên hiện tại – chi phí nghiệm tốt nhất đã biết), toàn bộ cây con tại nút này bị cắt tỉa.

Trong Branch and Price cho VRPTW, hàm đánh giá cận tại mỗi nút chính là **giá trị tối ưu của LP Relaxation** thu được từ Column Generation:

$$g(\text{nút}) = Z_{LP} = \min \sum_{r \in R} c_r y_r \quad \text{s.t.} \quad \sum_r a_{ir} y_r \geq 1, y_r \geq 0, \text{ ràng buộc phân nhánh}$$

Đây là cận dưới hợp lệ vì ba lý do:

- **LP Relaxation là cận dưới của ILP:** mọi nghiệm nguyên khả thi đều là nghiệm LP khả thi, nên $Z_{LP} \leq Z_{ILP}$.
- **Column Generation cho Z_{LP} chính xác:** dù RMP chỉ có tập con các cột, SPPRC đảm bảo không còn cột nào có reduced cost âm – tức RMP đã đạt tối ưu LP toàn phần.
- **Ràng buộc phân nhánh được nhúng vào:** việc chỉnh dist trực tiếp đảm bảo SPPRC chỉ sinh tuyến thỏa mãn ràng buộc phân nhánh, do đó Z_{LP} phản ánh đúng cận dưới của cây con hiện tại.

Điều kiện chặt nhánh: nút hiện tại bị loại bỏ khi:

$$Z_{LP} > \text{UB} = f^*$$

tức cận dưới LP đã vượt chi phí nghiệm nguyên tốt nhất hiện có – mọi nghiệm nguyên trong cây con này chắc chắn không tốt hơn f^* .

Chất lượng của hàm đánh giá cận: Trong thực tế, Z_{LP} từ Column Generation thường rất sát với Z_{ILP} , đặc biệt với VRPTW do cấu trúc Set Covering có tính chất totally unimodular gần đúng. Điều này giúp B&B cắt tỉa được phần lớn cây tìm kiếm ngay từ sớm, giảm đáng kể số nút cần xét so với Branch and Bound thuần túy không có Column Generation.

Cập nhật cận dưới (LB) và cận trên (UB)

Cận trên (UB) được khởi tạo là $+\infty$ và cập nhật mỗi khi tìm được nghiệm nguyên khả thi tốt hơn. Khi Column Generation hội tụ và ma trận edges hoàn toàn nguyên (mọi giá trị = 0 hoặc = 1), nghiệm được coi là khả thi nguyên. Nếu giá trị LP tại nút này nhỏ hơn UB hiện tại:

$$UB \leftarrow \text{branching.lowest_value}$$

và tập `best_routes` được cập nhật bằng các tuyến có $Q > 10^{-6}$.

Cận dưới (LB) được cập nhật tại các nút `toplevel` (nút cấp cao nhất của tìm kiếm theo chiều sâu). Cụ thể, khi cả hai nút con của nút cha đều đã được xử lý:

$$LB = \min(\text{son0.lowest_value}, \text{son1.lowest_value})$$

Đây là cận dưới LP chính xác: vì LP Relaxation cho cận dưới của bài toán nguyên, giá trị CG tại mỗi nút là cận dưới cục bộ cho toàn bộ cây con dưới nút đó.

Điều kiện cắt tỉa

Một nút bị loại bỏ (cắt tỉa) trong ba trường hợp:

1. **Gap đủ nhỏ:** $\frac{UB - LB}{UB} < \varepsilon = 10^{-6}$, kết thúc toàn bộ thuật toán.
2. **Relaxation vô nghiệm:** giá trị CG $> 2 \times \text{maxlength}$ (ước lượng trên cho bài toán khả thi) hoặc $< -10^{-6}$ (bất thường do lỗi số). Điều kiện này phát hiện các nhánh mà ràng buộc phân nhánh làm cho bài toán LP vô nghiệm hoặc không giới nội.
3. **Bound cut:** $\text{branching.lowest_value} > UB$, tức cận dưới LP tại nút này đã vượt cận trên toàn cục, mọi nghiệm nguyên trong cây con này đều không tốt hơn nghiệm tốt nhất đã biết.

3.6 Luồng thực thi tổng thể

Thuật toán: Branch and Price cho VRPTW

Input: user_param (đồ thị, ràng buộc VRPTW), init_routes

Output: best_routes, UB (chi phí tối ưu)

```

1:  $UB \leftarrow +\infty$ ,  $LB \leftarrow -\infty$ ,  $best\_routes \leftarrow \emptyset$ 
2: BB_NODE(user_param, init_routes, None, 0)

3: function BB_NODE(user_param, routes, branching, depth)
4:   if  $(UB - LB) / UB < \varepsilon$  then
5:     return ▷ Gap đủ nhỏ, kết thúc
6:   end if
7:   Áp đặt ràng buộc phân nhánh của nút này lên dist (không đệ quy)
8:    $(Z_{LP}, routes) \leftarrow \text{COLUMNGENERATION}(routes)$ 
9:   if  $Z_{LP} > 2 \times \text{maxlength}$  hoặc  $Z_{LP} < 0$  then
10:    return ▷ Relaxation vô nghiệm, cắt tỉa
11:  end if
12:  branching.lowest_value  $\leftarrow Z_{LP}$ 
13:  if branching.toplevel then
14:     $LB \leftarrow \min(\text{son0.lowest\_value}, Z_{LP})$ 
15:  end if
16:  if  $Z_{LP} > UB$  then
17:    return ▷ Bound cut, cắt tỉa
18:  end if
19:  Tính  $\text{edges}[i][j] \leftarrow \sum_r \text{route}.Q[r] \cdot \mathbf{1}[(i, j) \in r]$ 
20:  if mọi  $\text{edges}[i][j] \in \{0, 1\}$  then
21:    if  $Z_{LP} < UB$  then
22:       $UB \leftarrow Z_{LP}$ 
23:      Lưu best_routes từ các tuyến có  $Q > 10^{-6}$ 
24:    end if
25:  return ▷ Nghiệm nguyên, kết thúc nhánh

```

```

26:   else
27:       Chọn  $(i^*, j^*)$ :  $\arg \max \min(\text{edges}[i][j], |1 - \text{edges}[i][j]|) \times c_r$ 
28:       Xác định  $\text{best\_val} \in \{0, 1\}$  theo giá trị  $\text{edges}[i^*][j^*]$ 
        // Nhánh 1:  $\text{branch\_value} = \text{best\_val}$ 
29:        $\text{node}_1 \leftarrow \text{TREEBB}(\text{branching}, i^*, j^*, \text{best\_val})$ 
30:       Cập nhật dist theo  $\text{node}_1$  (không đệ quy)
31:        $R_1 \leftarrow$  tuyến không chứa cạnh  $(i^*, j^*)$ 
32:        $\text{BB\_NODE}(\text{user\_param}, R_1, \text{node}_1, \text{depth}+1)$ 
33:        $\text{branching.son0} \leftarrow \text{node}_1$ 
        // Nhánh 2:  $\text{branch\_value} = 1 - \text{best\_val}$ 
34:       Phục hồi dist  $\leftarrow \text{dist\_base}$ 
35:        $\text{node}_2 \leftarrow \text{TREEBB}(\text{branching}, i^*, j^*, 1 - \text{best\_val})$ 
36:       Tái áp dụng toàn bộ chuỗi ràng buộc từ gốc đến  $\text{node}_2$  (đệ quy)
37:        $R_2 \leftarrow$  tuyến có mọi cạnh không bị cấm trong dist
38:        $\text{BB\_NODE}(\text{user\_param}, R_2, \text{node}_2, \text{depth}+1)$ 
39:        $\text{branching.lowest\_value} \leftarrow \min(\text{node}_1.\text{lowest\_value}, \text{node}_2.\text{lowest\_value})$ 
40:   end if
41: end function

```

3.7 Độ phức tạp

- Gọi n là số khách hàng, $|R|$ là số cột trong RMP tại một nút.
- **Column Generation mỗi nút:** mỗi vòng lặp giải LP có $|R|$ biến và n ràng buộc, chi phí $O(n \cdot |R|^{1.5})$ (Simplex). SPPRC có độ phức tạp giả đa thức $O(2^n \cdot n^2)$ trong trường hợp xấu nhất do không gian trạng thái `vertex_visited`; tuy nhiên, dominance và kỹ thuật Feillet 2004 cắt giảm đáng kể số nhân thực tế.
- **Cây B&B:** số nút tối đa lý thuyết là $O(2^{n^2})$ theo số biến cạnh nhị phân x_{ij} . Trong thực tế, cận dưới LP chất lượng cao từ Column Generation cắt bỏ phần lớn cây, số nút thực tế nhỏ hơn nhiều.
- **Độ phức tạp tổng thể (worst case):**

$$T(n) = O\left(2^{n^2} \cdot 2^n \cdot n^4\right)$$

Trong thực nghiệm với các bộ Solomon (25–100 khách hàng), thuật toán hội tụ trong thời gian chấp nhận được nhờ chất lượng cận dưới của Column Generation và hiệu quả của SPPRC với dominance.

CHƯƠNG III: KẾT QUẢ THỰC NGHIỆM

1 Kết quả thực nghiệm

Ba thuật toán B&B, B&C và B&P được thực nghiệm trên ba bộ dữ liệu chuẩn C101 (Phân cụm), R101 (Ngẫu nhiên) và RC101 (Hỗn hợp), mỗi bộ dữ liệu được khảo sát ở ba mức quy mô $n \in \{5, 8, 11\}$ khách hàng.

Để đánh giá và phân tích sâu sắc đặc tính vận hành của ba thuật toán (Branch and Bound - B&B, Branch and Cut - B&C, và Branch and Price - B&P) trên cấu trúc cây tìm kiếm, hệ thống thực nghiệm sử dụng 10 tiêu chí đánh giá sau:

- **Min cost (Obj):** Tổng chi phí khoảng cách nhỏ nhất của hệ thống đạt được.
- **Optimality gap (Gap):** Tỷ lệ phần trăm chênh lệch giữa cận dưới toàn cục và nghiệm nguyên tốt nhất hiện tại ($\text{Gap} = \frac{|\text{Obj} - \text{LB}|}{\text{Obj}} \times 100\%$). Giá trị 0.00% chứng minh nghiệm tìm được đã tối ưu.
- **Time:** Tổng thời gian tới khi thuật toán tìm được nghiệm tối ưu, tính bằng giây (s).
- **Time / node (T.Node):** Tỷ lệ thời gian tính toán trung bình cho mỗi nút trên cây (ms/node), phản ánh độ phức tạp tính toán tại một bài toán con LP Relaxation.
- **Nodes:** Số lượng nút (bài toán nói lỏng tuyến tính) đã giải.
- **Branches:** Số lần phân nhánh.
- **Pruned:** Số lượng nút thực tế bị thuật toán bỏ qua do vi phạm điều kiện về cận dưới.
- **Pruning rate (P.Rate):** Tỷ lệ phần trăm số lượng nút bị cắt tĩa trên tổng số nút được sinh ra trên cây.
- **Infeasible nodes (Inf.):** Số lượng nút bị thuật toán loại bỏ do bài toán LP Relaxation tại nút đó vô nghiệm.
- **Integer solutions found (Int.):** Số lần thuật toán ghi nhận được một phương án phân tuyến đạt điều kiện nguyên (feasible) trong quá trình duyệt cây.

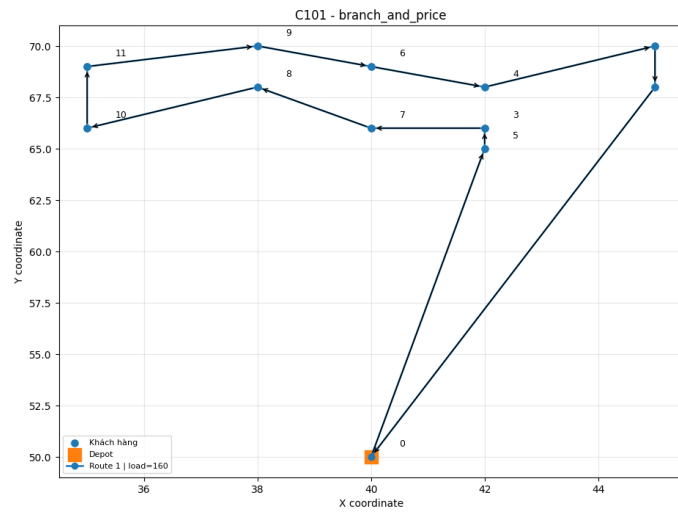
1.1 C101 (Phân cụm)

Bảng 1: Đánh giá hiệu năng tổng hợp của các thuật toán trên bộ dữ liệu **C101**

Tiêu chí đánh giá	n = 5			n = 8			n = 11		
(Metrics)	B&B	B&C	B&P	B&B	B&C	B&P	B&B	B&C	B&P
Obj	42.42	42.42	42.41	49.72	49.72	49.72	59.49	59.49	59.49
Gap (%)	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
Time (s)	0.03	0.03	0.01	1.03	0.13	0.07	473.92	77.80	0.50
T.Node (ms/node)	6.96	6.95	11.02	12.12	14.44	70.00	23.09	33.75	500.00
Nodes	5	5	1	85	9	1	20521	2305	1
Branches	2	2	0	84	8	0	20520	2304	0
Pruned	0	0	0	42	3	0	10132	1131	0
P.Rate (%)	0.0%	0.0%	100%	49.4%	43.8%	100%	50.0%	49.9%	100%
Inf. (Infeasible)	2	2	N/A	28	4	N/A	7122	864	N/A
Int. (Integer)	1	1	N/A	4	2	N/A	12	5	N/A



Hình 2: Biểu đồ so sánh thời gian chạy của các thuật toán (C101)

Hình 3: Sơ đồ lộ trình tối ưu tại mốc $n = 11$ (C101)

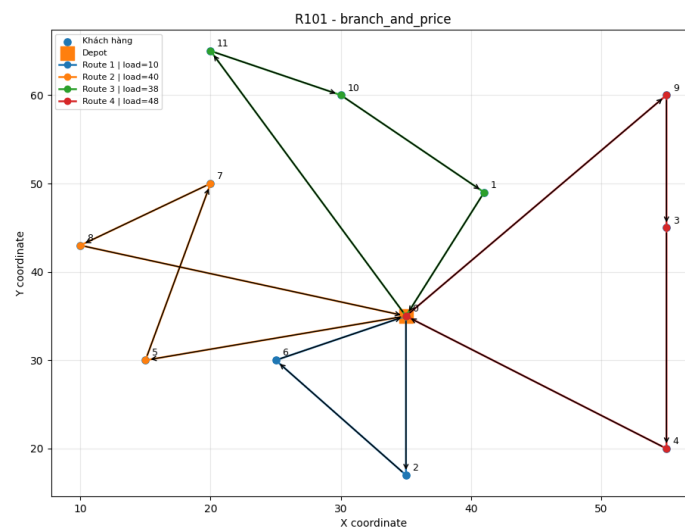
1.2 R101 (Ngẫu nhiên)

Bảng 2: Đánh giá hiệu năng tổng hợp của các thuật toán trên bộ dữ liệu **R101**

Tiêu chí đánh giá	n = 5			n = 8			n = 11		
(Metrics)	B&B	B&C	B&P	B&B	B&C	B&P	B&B	B&C	B&P
Obj	156.35	156.35	156.35	216.84	216.84	216.84	297.79	297.79	297.79
Gap (%)	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
Time (s)	0.07	0.21	0.01	7.29	6.12	0.01	850.20	1239.45	0.02
T.Node (ms/node)	5.76	7.62	6.28	11.81	13.68	8.18	21.18	35.80	22.66
Nodes	13	27	1	617	447	1	40135	34623	1
Branches	6	13	0	308	223	0	20067	17311	0
Pruned	0	3	0	90	79	0	8722	7306	0
P.Rate (%)	0.0%	11.1%	100%	14.6%	17.7%	100%	21.7%	21.1%	100%
Inf. (Infeasible)	6	9	N/A	218	144	N/A	11342	10003	N/A
Int. (Integer)	1	2	N/A	1	1	N/A	4	3	N/A



Hình 4: Biểu đồ so sánh thời gian chạy của các thuật toán (R101)



Hình 5: Sơ đồ lộ trình tối ưu tại mốc $n = 11$ (R101)

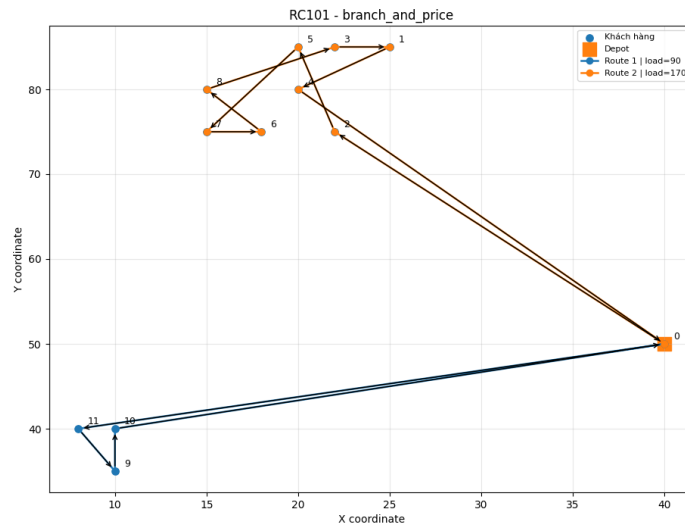
1.3 RC101 (Hỗn hợp)

Bảng 3: Đánh giá hiệu năng tổng hợp của các thuật toán trên bộ dữ liệu **RC101**

Tiêu chí đánh giá	n = 5			n = 8			n = 11		
(Metrics)	B&B	B&C	B&P	B&B	B&C	B&P	B&B	B&C	B&P
Obj	89.13	89.13	89.13	115.74	115.74	115.74	191.28	191.28	191.28
Gap (%)	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
Time (s)	0.27	0.19	0.01	19.27	23.50	0.16	981.38	140.98	0.13
T.Node (ms/node)	5.57	6.41	12.02	15.18	18.20	162.76	21.49	30.23	131.98
Nodes	49	29	1	1269	1291	1	45667	4663	1
Branches	24	14	0	634	645	0	22833	2331	0
Pruned	7	3	0	169	190	0	8117	965	0
P.Rate (%)	14.3%	10.3%	100%	13.3%	14.7%	100%	17.8%	20.7%	100%
Inf. (Infeasible)	14	9	N/A	462	447	N/A	14712	1355	N/A
Int. (Integer)	4	3	N/A	4	9	N/A	5	12	N/A



Hình 6: Biểu đồ so sánh thời gian chạy của các thuật toán (RC101)



Hình 7: Sơ đồ lộ trình tối ưu tại mốc $n = 11$ (RC101)

2 Đánh giá chi tiết

Thông qua việc đối chiếu kết quả thực nghiệm trên ba bộ dữ liệu C101 (Phân cụm), R101 (Ngẫu nhiên) và RC101 (Hỗn hợp), ta rút ra một số nhận xét sau:

- **Branch and Price (B&P)** là thuật toán chạy nhanh nhất trên cả ba bộ dữ liệu ở mọi mốc quy mô. Đặc điểm nổi bật là B&P luôn chỉ cần giải đúng một nút duy nhất (Nodes = 1, Branches = 0), tức là cơ chế sinh cột (Column Generation) đã tạo ra nghiệm nguyên tối ưu ngay tại nút gốc mà không cần phân nhánh, dẫn đến tỷ lệ cắt tĩa luôn đạt 100%. Về thời gian, tại mốc $n = 11$ B&P hoàn thành trong 0.50s (C101), 0.02s (R101) và 0.13s (RC101), nhanh hơn B&B lần lượt khoảng 948 lần, 42.500 lần và 7.549 lần. Tuy nhiên, thời gian xử lý trung bình mỗi nút (T.Node) của B&P cao hơn đáng kể so với B&B và B&C — ví dụ tại C101 ($n = 11$) là 500.00 ms/node so với 23.09 và 33.75 ms/node — phản ánh chi phí giải bài toán con định giá (pricing) tại mỗi lần sinh cột.
- **Branch and Cut (B&C)** cho thấy hiệu quả phụ thuộc rõ ràng vào đặc tính phân bố dữ liệu:
 - Trên dữ liệu Phân cụm (C101) và Hỗn hợp (RC101): Các ràng buộc cắt phát huy tác dụng tốt, giúp B&C giải ít nút hơn B&B đáng kể. Tại C101 ($n = 11$), B&C chỉ

cần duyệt 2.305 nút so với 20.521 nút của B&B (giảm $\approx 88.8\%$), tìm được 5 nghiệm nguyên so với 12 của B&B, và hoàn thành trong 77.80s thay vì 473.92s. Tại RC101 ($n = 11$), số nút giảm từ 45.667 xuống còn 4.663 (giảm $\approx 89.8\%$), thời gian giảm từ 981.38s xuống 140.98s.

- *Trên dữ liệu Ngẫu nhiên (R101)*: Các ràng buộc cắt kém hiệu quả hơn do khách hàng phân bố rời rạc không theo quy luật. Tại R101 ($n = 11$), dù B&C duyệt ít nút hơn B&B (34.623 so với 40.135, giảm $\approx 13.7\%$) và tỷ lệ cắt tĩa tương đương (21.1% so với 21.7%), tổng thời gian thực thi của B&C lại cao hơn (1239.45s so với 850.20s của B&B, chậm hơn $\approx 45.8\%$). Nguyên nhân là T.Node của B&C tại R101 ($n = 11$) đạt 35.80 ms/node, cao hơn B&B (21.18 ms/node), cho thấy chi phí sinh và kiểm tra các ràng buộc cắt trên cấu trúc ngẫu nhiên vượt quá lợi ích mà chúng mang lại.
- **Branch and Bound (B&B)** không có cơ chế hỗ trợ nào ngoài phân nhánh và cắt tĩa theo cận, nên số nút và số nhánh tăng nhanh theo quy mô. Tại $n = 11$, B&B phải duyệt lần lượt 20.521 nút (C101), 40.135 nút (R101) và 45.667 nút (RC101), với số nút vô nghiệm chiếm tỷ lệ lớn (7.122, 11.342 và 14.712 nút). Tỷ lệ cắt tĩa của B&B dao động quanh 50% trên dữ liệu phân cụm (C101) nhưng chỉ đạt $\approx 14\text{--}22\%$ trên dữ liệu ngẫu nhiên và hỗn hợp, cho thấy cận dưới của B&B đơn thuần không đủ chặt để loại bỏ sớm các nhánh không triển vọng.

Nhìn chung, cả B&B và B&C đều gặp giới hạn thực tế ở quy mô $n \leq 11$ do thời gian chạy tăng theo cấp số nhân — tại $n = 11$ một số cấu hình đã vượt quá 900 giây. Trong khi đó, B&P có thể mở rộng lên đến $n = 100$ khách hàng nhờ cơ chế sinh cột kiểm soát được kích thước không gian nghiệm.

Bảng tổng kết so sánh ba thuật toán

Bảng 4: Tổng kết so sánh đặc điểm vận hành của B&B, B&C và B&P

Tiêu chí	B&B	B&C	B&P
Chất lượng nghiệm (Gap)	Tối ưu	Tối ưu	Tối ưu
Thời gian hội tụ	Chậm nhất (473s – 981s)	Trung bình (78s – 1239s)	Nhanh nhất (0.02s – 0.50s)
Số nút cần duyệt	Nhiều nhất (20.521 – 45.667)	Trung bình (2.305 – 34.623)	1 nút
Tỷ lệ cắt tỉa	18% – 50%	20% – 50%	100%
Chi phí mỗi nút (T.Node)	Thấp nhất (21 – 23 ms)	Trung bình (34 – 36 ms)	Cao nhất (22 – 500 ms)
Hiệu quả trên dữ liệu phân cụm (C)	Kém	Tốt	Tốt nhất
Hiệu quả trên dữ liệu ngẫu nhiên (R)	Trung bình	Kém hơn B&B	Tốt nhất
Hiệu quả trên dữ liệu hỗn hợp (RC)	Kém	Tốt	Tốt nhất
Số nút vô nghiệm	Nhiều nhất (7.122 – 14.712)	Trung bình (864 – 10.003)	Không có (N/A)
Số nghiệm nguyên tìm được ($n = 11$)	4 – 12	3 – 12	N/A
Khả năng mở rộng	$n \leq 11$	$n \leq 11$	$n \leq 100$

KẾT LUẬN

Bài báo cáo đã triển khai và so sánh thực nghiệm ba thuật toán giải chính xác cho bài toán định tuyến phương tiện có ràng buộc thời gian (VRPTW): Branch and Bound (B&B), Branch and Cut (B&C) và Branch and Price (B&P), trên ba bộ dữ liệu chuẩn với các đặc tính phân bố không gian khác nhau (C101, R101, RC101) ở các mức quy mô nhỏ $n \in \{5, 8, 11\}$.

Kết quả thực nghiệm cho thấy cả ba thuật toán đều tìm được nghiệm tối ưu (Gap = 0.00%) trên toàn bộ các cấu hình thử nghiệm, tuy nhiên chi phí tính toán để đạt được nghiệm đó có sự khác biệt lớn. B&P là thuật toán hiệu quả nhất về thời gian và khả năng mở rộng, có thể xử lý bài toán đến $n = 100$ khách hàng nhờ cơ chế sinh cột kiểm soát được kích thước không gian tìm kiếm. B&C hoạt động tốt trên dữ liệu có cấu trúc phân cụm nhưng kém hiệu quả trên dữ liệu ngẫu nhiên do chi phí quản lý các ràng buộc cắt vượt quá lợi ích chúng mang lại. B&B đơn thuần không đủ khả năng kiểm soát sự phát triển của cây tìm kiếm khi quy mô tăng, và cả hai thuật toán B&B lẫn B&C đều gặp giới hạn thực tế ở $n \leq 11$.

Hướng phát triển tiếp theo có thể tập trung vào việc tích hợp các chiến lược khởi tạo nghiệm ban đầu (warm start) hoặc kết hợp B&P với các phương pháp heuristic để rút ngắn thêm thời gian hội tụ trên các bài toán thực tế có quy mô lớn hơn.

Tài liệu tham khảo

- [1] J. F. Bard, G. Kontoravdis **and** G. Yu, “A Branch-and-Cut Procedure for the Vehicle Routing Problem with Time Windows,” *Transportation Science*, **jourvol** 36, **number** 2, **pages** 250–269, 2002. DOI: 10.1287/trsc.36.2.250.565.
- [2] B. Kallehauge, “On the Vehicle Routing Problem with Time Windows,” phdthesis, Technical University of Denmark, Kgs. Lyngby, Denmark, 2006.